BiteBuddy

Group Dining Made Easy

Project Report

Team Name: Group 7

Team Members

Neon Zheng	Rafid Nasery
Jonathan Lee	Abdala Aljewarane
Billy Nguyen	Paola Reyes-Almendares

Supervisor: Professor Sridhar Alagar

Project Coordinator: Adarsh Gella

Erik Jonsson School of Engineering and Computer Science
University of Texas at Dallas

May 1, 2026

Contents

Executive Summary **2**

1 Introduction **3**
1.1 Motivation & Problem Statement . . . 3
1.2 Objectives, Scope, and Goals Achieved . . . 3
1.3 Proposed Solution . . . 4
1.4 Key Highlights . . . 5

2 Project Specification & Management **6**
2.1 The Project Specification (The AI's Blueprint) . . . 6
2.2 Maintaining the "Single Source of Truth" . . . 6
2.3 Key Milestones . . . 6
2.4 Team Roles & Leader Rotation . . . 7

3 System Architecture & High-Level Design **8**
3.1 System Overview . . . 8
3.2 Architecture Diagrams . . . 9
3.3 Key Architectural Decisions and Trade-offs . . . 12

4 Implementation & AI Orchestration **13**
4.1 Technologies Used . . . 13
4.2 AI Build Strategy . . . 14
4.3 Human-in-the-Loop Interventions . . . 15
4.4 Codebase . . . 16

5 System Performance & Validation **16**
5.1 Testing and Validation . . . 16
5.2 Technical Performance Metrics . . . 18
5.3 AI Model Performance . . . 18

6 Lessons Learned **19**
6.1 Insights Gained . . . 19
6.2 Lessons from Challenges . . . 19
6.3 Skills Developed . . . 20
6.4 Individual Reflection . . . 20

7 Conclusion & Future Work **22**
7.1 Summary of Key Accomplishments . . . 22
7.2 Future Enhancements . . . 22

References **23**

Acknowledgements **24**

Executive Summary

BiteBuddy is a mobile group dining decision application that eliminates the frustration of choosing where to eat with friends. Users create shared sessions, swipe on nearby restaurants, and receive an instant match notification the moment every member of the group likes the same place. Built as a full-stack monorepo — a Next.js 16 REST API, an Expo 54 React Native mobile client, and a shared TypeScript package — BiteBuddy leverages Supabase for authentication, database persistence, row-level security, and real-time event delivery, and the Google Places API (New) for live restaurant discovery.

Match detection is performed atomically inside a PostgreSQL trigger, eliminating race conditions without any app-layer polling. Over the course of the semester the team produced a fully functional MVP covering user authentication, friend management, location-aware session creation, collaborative swiping, real-time lobby synchronization, and a results screen showing matched restaurants ranked by group consensus.

The project was planned and implemented with AI coding assistants (Claude Code) serving as primary engineering agents, with team members acting as architects, reviewers, and testers throughout.

1. Introduction

1.1 Motivation & Problem Statement

Deciding where a group of friends should eat is one of the most universally recognized social friction points. The typical decision cycle — a group chat thread that spirals into "I don't care, you pick," followed by suggestions that are immediately vetoed, followed by a drawn-out negotiation — wastes time and often ends in a compromised choice that satisfies no one. The problem compounds at scale: the larger the group, the longer the process.

Existing solutions either solve the solo-dining discovery problem (Yelp, Google Maps) or the group-coordination problem (shared lists, polls), but none merge real-time restaurant discovery with synchronous group preference voting in a friction-free, gamified interface. We identified an opportunity to apply the swipe-to-decide UX pattern — proven effective by Tinder and similar apps — to the restaurant selection problem, with the critical addition that the decision is made collectively and anonymously, removing social pressure to conform to dominant voices in the group.

The core insight is that a match should only surface when *every* person in the group has independently liked a restaurant — not merely voted in a poll, which is susceptible to anchoring bias. This guarantees the result is a genuine group preference.

1.2 Objectives, Scope, and Goals Achieved

Intended Functionalities

- User registration and authentication with persistent sessions
- User profiles with display names, usernames, and avatar photos
- Friend system: search by username, send/receive/accept/decline friend requests
- Session creation with geolocation, search radius, price filter, and cuisine category filter
- Invite friends to sessions and join sessions via 6-character alphanumeric invite codes
- Real-time lobby showing which invited members have joined before swiping begins
- Per-session restaurant feed populated from Google Places API, deduplicated across sessions
- Card-swipe interface (like / dislike) on each restaurant with photo, name, rating, price level, and address
- Atomic server-side match detection when all members like the same restaurant
- Real-time match notification delivered to all swipe screens via Supabase Realtime
- Results screen listing all matches, ranked by consensus
- Session history listing past completed sessions and their matches
- Recent matches view across all sessions

Scope

The project is a cross-platform mobile application (iOS and Android via Expo) backed by a serverless Next.js API deployed on Vercel-compatible infrastructure. The database is a managed Supabase PostgreSQL instance. The scope did not include a web frontend, push notifications via APNs/FCM, payment integration, restaurant reservation booking, or in-app messaging.

Engineering Goals Successfully Completed

- Full authentication flow (signup, login, JWT refresh, logout, profile setup, password reset)
- 20+ REST API endpoints with JWT-based auth and Supabase RLS on every route
- Friend management system with directional requests and search
- Session lifecycle: create → invite → lobby → swipe → complete/cancelled
- Google Places API integration with price-level and category filtering
- Atomic match detection via PostgreSQL trigger (zero race conditions)
- Session auto-completion via `maybe_complete_session()` database function
- Supabase Realtime subscriptions for lobby presence and swipe progress
- Centroid-based location averaging so session search origin reflects all members' positions
- Haversine distance calculation in shared utility layer
- Avatar upload via Supabase Storage
- 13 migration files establishing full schema with RLS policies, indexes, and RPCs
- Vitest unit test coverage for the Google Places client and shared type utilities
- TypeScript end-to-end with a shared package consumed by both API and mobile without a build step

1.3 Proposed Solution

BiteBuddy is structured as a three-package npm workspace monorepo. The API (`apps/api`) is a Next.js 16 App Router application whose file-based routing defines all REST endpoints. Every protected route begins by calling `getAuthenticatedClient(request)`, which extracts the Bearer JWT sent by the mobile client, constructs an authenticated Supabase client (so RLS policies automatically scope all queries to the requesting user), and returns the verified user record. Restaurant data is fetched from the Google Places (New) Nearby Search API and stored denormalized into the `session_restaurants` table at session-start time, so subsequent swipe requests are purely database operations with no external API calls on the hot path.

The mobile client (`apps/mobile`) is built with Expo 54 and React Native 0.81, navigated by Expo Router (file-based, same mental model as Next.js). Supabase Auth is called directly

from the mobile client; the resulting JWT is attached as an `Authorization: Bearer` header on every call to the API. Real-time features (lobby presence, swipe progress counters, match notifications) are delivered via Supabase Realtime channel subscriptions established directly from the mobile client, avoiding polling and reducing API load.

The shared package (`packages/shared`) exports TypeScript interfaces for all database entities and API request/response shapes, a `CUISINE_CATEGORIES` constant, and geolocation utilities (`calculateCentroid`, `calculateDistance`). It is consumed as raw TypeScript by both the API and the mobile client, ensuring type safety across the full stack without a separate compilation step.

Match detection is intentionally placed inside the database layer. The `check_for_match()` trigger fires on every row inserted into the `swipes` table. It counts how many members of the session have liked the restaurant just swiped on; if that count equals the total member count, it inserts a row into the `matches` table inside the same transaction. This atomicity means concurrent swipes from different users on the same restaurant cannot produce duplicate matches regardless of network timing.

1.4 Key Highlights

Technology Breakthroughs

- **Atomic match detection via PostgreSQL trigger** — By placing consensus logic inside a database transaction, BiteBuddy avoids the classic "last swipe race condition" that would arise from application-level match checking.
- **Centroid-based session origin** — Rather than using only the session creator's location, the API stores all members' reported coordinates in a JSONB column and averages them into a centroid before querying Google Places, so restaurant results are geographically fair to every participant.
- **Zero-build shared types** — Sharing TypeScript interfaces between a Next.js server and an Expo client without a compilation or publishing step, using `transpilePackages` and workspace references, eliminated an entire class of type-drift bugs.
- **Invite code system** — A 6-character alphanumeric invite code on each session enables deep-linking and out-of-band sharing (copy/paste in any messenger) without requiring friends to already be connected in the app.

Key Findings

- Supabase Realtime channels proved sufficient for all live-update requirements, removing the need for a separate WebSocket server.
- Google Places (New) API's field-mask pattern (`X-Goog-FieldMask` header) allows precise control over billed fields, keeping API costs proportional to actual data consumed.
- Row-level security policies on every table meant that authorization bugs were caught at the database layer rather than requiring application-level guards on every query.

- Expo Router's file-based navigation closely mirrored Next.js conventions, enabling team members to context-switch between frontend and backend with minimal friction.

2. Project Specification & Management

2.1 The Project Specification (The AI's Blueprint)

The BiteBuddy project spec is an "Agent-Ready" document maintained in the team's shared Google Drive. It serves as the authoritative instruction set passed to Claude Code at the start of each engineering session. The spec is structured into discrete, machine-readable sections: a system overview, table-by-table database schema with column types and constraints, endpoint-by-endpoint API contract (method, path, authentication requirement, request body shape, response shape, error codes), mobile screen inventory with navigation flow, and a list of explicitly out-of-scope items to prevent scope creep by the AI agent.

The spec also includes a `CLAUDE.md` file committed directly into the repository root, which Claude Code automatically reads at session start. This file describes the monorepo structure, common commands, environment variable conventions, and the key architectural patterns (Bearer JWT flow, RLS, Realtime, trigger-based match detection) so that AI agents operating in the codebase have immediate architectural context without requiring the team to re-explain the system each session.

Link to live spec document: (To be inserted by team — Google Drive link)

2.2 Maintaining the "Single Source of Truth"

The spec was treated as a living document under the same discipline as code: changes to requirements were reflected in the spec *before* implementation began, not after. The team enforced a rule that any architectural decision made during an AI-assisted session that diverged from the spec had to be reconciled in one of two ways — either the AI's output was corrected to match the spec, or the spec was deliberately updated to reflect a consciously chosen new direction.

Version control for the spec was managed through Google Drive's revision history combined with Git commits to `CLAUDE.md`. When a database migration was merged, the corresponding table definition in the spec was updated in the same PR description. The `CLAUDE.md` file was also updated incrementally — for example, when the team switched the external restaurant API from Yelp to Google Places (New), `CLAUDE.md` was updated to note `lib/yelp.ts` was kept for historical reasons but wraps the Google Places API, preventing future AI sessions from making incorrect assumptions about the data provider.

2.3 Key Milestones

Weeks 1–2 Specification & Architecture: The team produced the initial project spec, settled on the monorepo structure, and selected the technology stack. The Supabase project was provisioned and the first two migration files (profiles and friendships) were written and tested.

Weeks 3–4 **Core Infrastructure:** Authentication endpoints were completed and integrated with the Expo client. The `AuthContext` on mobile was established with JWT auto-refresh. Friend request and search endpoints went live. The shared TypeScript package was wired up across both apps.

Weeks 5–6 **Session MVP:** Session creation, invite codes, the lobby screen with Real-time presence, and the Google Places integration were completed. This represented the first end-to-end flow a real user could exercise.

Weeks 7–8 **Swipe Engine & Match Detection:** The swipe endpoint, the `check_for_match()` trigger, and the real-time match notification on the swipe screen were implemented. The `maybe_complete_session()` RPC was added to auto-complete sessions when all members finished swiping. This was the most technically complex phase.

Weeks 9–10 **Polish & Performance:** Performance indexes (migration 013) were added. Avatar upload via Supabase Storage, the history tab, recent matches endpoint, session cancellation, and the leave-session flow were completed. The test suite was written for the Google Places client and shared utilities. The `CLAUDE.md` and spec were brought up to date.

Week 11 **Final Integration & Report:** End-to-end testing across two physical devices, bug fixes, and preparation of this report.

2.4 Team Roles & Leader Rotation

The team rotated the "team leader" role every two weeks, giving each member ownership of sprint planning, PR review coordination, and spec update responsibility for their rotation.

Table 1: Leader Rotation Schedule

Rotation	Leader	Period
1	Rafid Nasery	Weeks 1–2
2	Neon Zheng	Weeks 3–4
3	Jonathan Lee	Weeks 5–6
4	Billy Nguyen	Weeks 7–8
5	Abdala Aljewarane	Weeks 9–10
6	Paola Reyes-Almendares	Week 11

Table 2: Technical Responsibilities by Team Member

Member	Primary Responsibilities
Rafid Nasery	Monorepo setup, API architecture, authentication endpoints, Supabase configuration, `CLAUDE.md` maintenance
Neon Zheng	Session CRUD endpoints, Google Places API integration (`lib/yelp.ts`), session-restaurants pipeline
Jonathan Lee	Swipe endpoint, match-detection trigger, `maybe_complete_session()` RPC, results endpoint
Billy Nguyen	Mobile navigation (Expo Router), auth screens, `AuthContext`, `lib/api.ts` helper layer
Abdala Aljewarane	Swipe UI, Realtime subscriptions (lobby + swipe + match), real-time match notification
Paola Reyes-Almendares	Friends screens, profile setup, avatar upload, history tab, session creation UI

3. System Architecture & High-Level Design

3.1 System Overview

BiteBuddy is a client–server system composed of three primary components: a REST API, a mobile client, and a managed cloud database. All communication between the mobile client and the API is over HTTPS using JSON. Authentication tokens are issued by Supabase Auth and forwarded by the mobile client as `Authorization: Bearer <jwt>` headers on every API request. The API validates these tokens by constructing a Supabase client with the user's JWT, which simultaneously authenticates the request and scopes all database queries through Supabase Row Level Security policies.

Key API Endpoints

Method	Path	Description
POST	`/api/auth/signup`	Register new user
POST	`/api/auth/login`	Authenticate, receive JWT
GET	`/api/auth/me`	Fetch authenticated user profile
POST	`/api/auth/logout`	Invalidate session
PUT	`/api/profile`	Update display name, username, avatar
GET	`/api/friends`	List accepted friends
GET	`/api/friends/requests`	List incoming friend requests

Method	Path	Description
POST	`/api/friends/request`	Send friend request by username
POST	`/api/friends/respond`	Accept or decline a friend request
GET	`/api/friends/search`	Search users by username
GET	`/api/sessions`	List user's sessions (filterable by status)
POST	`/api/sessions`	Create a new session
GET	`/api/sessions/[id]`	Get session details with member list
POST	`/api/sessions/[id]/start`	Fetch restaurants from Google Places, transition to active
POST	`/api/sessions/[id]/swipe`	Submit a swipe (like/dislike)
GET	`/api/sessions/[id]/restaurants`	List restaurants in the session
GET	`/api/sessions/[id]/results`	Get matches and swipe progress
POST	`/api/sessions/[id]/invite`	Invite friends to session
POST	`/api/sessions/join-by-code`	Join a session via 6-char invite code
GET	`/api/sessions/recent-matches`	Recent matches across all sessions
GET	`/api/health`	Health check

Table 3: REST API Endpoint Reference

Full API documentation is available in the repository at `apps/api/README.md`.

3.2 Architecture Diagrams

High-Level Architecture

```
+-------------------------------------------------------------+
|                     Mobile Client (Expo)                     |
|  +----------+  +--------------+  +------------------------+ |
|  |Auth Flow |  | Expo Router  |  | Supabase Realtime      | |
|  |(Supabase)|  |  Screens     |  | (lobby, swipe, matches)| |
|  +----+-----+  +------+-------+  +------------+-----------+ |
|       |               |                       |             |
|       +---------------+--------+--------------+             |
|                                | Bearer JWT                  |
+--------------------------------|---------+-------------------+
                                 | HTTPS / REST
                                 v
```

```
+-------------------------------------------------------------+
|                    Next.js 16 API (Vercel)                   |
|  +----------------+  +--------------+  +-----------------+  |
|  |  lib/auth.ts   |  |  Route       |  |  lib/yelp.ts    |  |
|  |  JWT Validate  |  |  Handlers    |  |  Google Places  |  |
|  +----------------+  +------+-------+  +--------+--------+  |
+------------------------------|-----------------|-----------+
                               |                 |
                   Supabase JS Client     Google Places API
                               |
                               v
+-------------------------------------------------------------+
|                       Supabase Cloud                         |
|  +--------------+  +-----------------+  +---------------+  |
|  | PostgreSQL   |  | Auth (GoTrue)   |  | Realtime      |  |
|  | + RLS        |  |                 |  | (WebSocket)   |  |
|  | + Triggers   |  +-----------------+  +---------------+  |
|  | + RPCs       |                                           |
|  +--------------+  +-----------------+                      |
|                    | Storage (avatars)|                      |
|                    +-----------------+                      |
+-------------------------------------------------------------+
```

Data Flow — Swipe & Match Detection

```
Mobile Client                    API                     PostgreSQL
     |                            |                          |
     |  POST /sessions/[id]/swipe|                          |
     |  { restaurant_id, liked } |                          |
     |-------------------------->|                          |
     |                            |  INSERT INTO swipes      |
     |                            |------------------------->|
     |                            |                          | TRIGGER
   fires:
     |                            |                          |
   check_for_match()
     |                            |                          |   COUNT(
   likes) == COUNT(members)?
     |                            |                          |   YES ->
   INSERT INTO matches
     |                            |                          |
     |                            |  maybe_complete_session()|
     |                            |------------------------->|
     |                            |                          | all members
    done?
     |                            |                          | YES ->
   status='completed'
     |                            |                          |
     |                            |<-------------------------|
     |<---------------------------|                          |
     |  { swipe_id, is_match,     |                          |
     |    match? }                |                          |
     |                            |                          |
```

```
     | (Supabase Realtime delivers match event to ALL      |
     |  connected clients on the session channel)          |
```

Database ER Diagram (Simplified)

```
profiles
  id (PK)
  username (UNIQUE)
  display_name
  avatar_url

friendships
  requester_id --> profiles.id
  addressee_id --> profiles.id
  status (pending|accepted|declined)

sessions
  id (PK)
  created_by --> profiles.id
  status (waiting|active|completed|cancelled)
  latitude, longitude, radius_meters
  price_filter[], category_filter
  invite_code (UNIQUE)
  user_locations (JSONB)

session_members
  session_id --> sessions.id
  user_id    --> profiles.id

session_restaurants
  session_id --> sessions.id
  yelp_id (Google Place ID)
  name, address, rating, price, categories...

swipes
  session_id    --> sessions.id
  user_id       --> profiles.id
  restaurant_id --> session_restaurants.id
  liked (BOOLEAN)
  UNIQUE(session_id, user_id, restaurant_id)

matches
  session_id    --> sessions.id
  restaurant_id --> session_restaurants.id
  matched_at
```

Mobile Navigation Structure

```
Root Layout
+-- (auth)/
|   +-- index.tsx         <- Landing / splash
|   +-- login.tsx
```

```
|   +-- signup.tsx
|   +-- profile-setup.tsx
|   +-- forgot-password.tsx
|   +-- reset-password.tsx
+-- (tabs)/
    +-- index.tsx          <- Home / active sessions
    +-- sessions.tsx       <- My sessions list
    +-- friends.tsx        <- Friends + requests
    +-- profile.tsx        <- User profile
    +-- history.tsx        <- Completed sessions
        session/[id]/
        +-- create.tsx     <- Create session (from home)
        +-- lobby.tsx      <- Waiting room (Realtime)
        +-- swipe.tsx      <- Card swipe interface (Realtime)
        +-- results.tsx    <- Matches + progress
```

3.3 Key Architectural Decisions and Trade-offs

Decision 1: Trigger-based vs. Application-level Match Detection

We placed the match consensus check inside a PostgreSQL trigger (`check_for_match()`) rather than in the API route handler. The trade-off is that database-layer logic is harder to unit test and debug than application code. However, the correctness guarantee is far stronger: because the trigger runs inside the same transaction as the swipe insert, two simultaneous swipes that both complete a match can never produce two match records. Any application-level approach would require distributed locking or an upsert pattern that is significantly more complex. For a feature that is central to the product's value proposition, correctness was chosen over testability.

Decision 2: Denormalized Restaurant Data per Session

Restaurants fetched from Google Places are stored in a per-session `session_restaurants` table rather than a shared `restaurants` table. This means the same physical restaurant can exist in multiple sessions as separate rows. The trade-off is storage duplication, but the benefit is that each session's restaurant set is immutable after `start` is called — later Google Places API changes or restaurant closures cannot retroactively alter a session's data. It also simplifies the swipe foreign-key chain: `swipes.restaurant_id` always points to a row that belongs to the same session.

Decision 3: Supabase Realtime vs. Polling

Rather than polling the API every few seconds for lobby state or swipe progress, the mobile client subscribes directly to Supabase Realtime channels. This eliminates artificial latency, reduces API server load, and provides instant match delivery. The trade-off is that the mobile client has two backend dependencies (the API and Supabase directly) rather than one, increasing coupling. We accepted this trade-off because Supabase Realtime is already a dependency for Auth, so no new vendor relationship was introduced.

Decision 4: Shared TypeScript Package Without a Build Step

The `packages/shared` package exports raw TypeScript source. The API consumes it via `transpilePackages` in `next.config.ts`; the Expo client consumes it via the Metro bundler's TypeScript support. This eliminates the build-and-publish loop that would be required if the package emitted compiled JavaScript. The trade-off is that both consumers must be able to transpile TypeScript, which is true of both Next.js and Expo but would not be true of a plain Node.js consumer.

Decision 5: Next.js 16 App Router for the API

Using Next.js as a pure API server (no server-rendered pages) trades a small amount of cold-start overhead (Vercel serverless functions) for the benefit of a well-understood deployment target, built-in TypeScript support, and file-based routing that keeps endpoint organization self-documenting. An alternative such as Fastify or Hono would have lower latency, but Next.js's ecosystem fit with Supabase JS and Vercel deployment was deemed more valuable at this project's scale.

4. Implementation & AI Orchestration

4.1 Technologies Used

Technology	Version	Role
Next.js	16.1.6	API server (App Router, file-based REST routes, TypeScript transpilation of shared package)
React	19.1.0	React runtime for both Next.js and Expo
Expo	54.0.33	Mobile app toolchain: build system, dev server, OTA updates
React Native	0.81.5	Cross-platform mobile UI primitives
Expo Router	4.0.0	File-based navigation for the mobile app (mirrors Next.js App Router conventions)
TypeScript	5.8.x	Static typing across all three packages
Supabase JS	2.49.x	Auth, database client, Storage, and Realtime — used on both API and mobile

Technology	Version	Role
PostgreSQL (via Supabase)	15	Primary database with RLS, triggers, and custom RPC functions
Google Places API (New)	v1	Nearby restaurant search; `places:searchNearby` endpoint with field-mask billing
Expo Location	18.0.0	Device GPS access for session origin coordinates
Expo Image Picker	17.0.10	Avatar photo selection and upload
React Native Reanimated	3.17.0	GPU-accelerated card swipe animations
React Native Gesture Handler	2.24.0	Touch gesture recognition for swipe cards
AsyncStorage	2.1.2	Persistent JWT storage on mobile
Vitest	2.0.0	Unit testing framework for API and shared package
npm workspaces	(built-in)	Monorepo dependency management and script orchestration

Table 4: Technology Stack

4.2 AI Build Strategy

The primary AI engineering tool used throughout the project was Claude Code (Anthropic), a CLI-based agentic coding assistant. The team's strategy was to treat Claude Code as a senior engineer who could write complete feature implementations, provided it was given a precise and unambiguous specification.

System Persona & Instruction Logic

The core instruction layer was the `CLAUDE.md` file committed to the repository root. This file established:

- The monorepo structure and which package owns which concern
- The authentication pattern every API route must follow (`getAuthenticatedClient` first, always)
- That RLS is the authorization layer (never write application-level user-ID filtering as a

substitute)

- That match detection lives in the database, not in the route handler
- Common commands for running each workspace
- Environment variable naming conventions

By establishing these constraints in `CLAUDE.md`, the team avoided having to re-specify them in every session prompt.

Prompt Patterns

For new features, the team followed a structured prompt pattern:

1. **Context anchoring** — "We are adding the `POST /api/sessions/[id]/invite` endpoint to the BiteBuddy API. Follow the auth pattern in `lib/auth.ts` and match the response shape defined in `packages/shared/src/api.ts`."
2. **Explicit schema reference** — Migration file names were cited directly so the agent could read the exact column names rather than inferring them.
3. **Constraint enumeration** — "Only invited friends (already in the `friendships` table with `status='accepted'`) may be added. Return 400 if a user is already a member."
4. **Output format** — "Write the route handler in `apps/api/src/app/api/sessions/[id]/invite/route` Also write the corresponding mobile API call in `lib/api.ts`."

Prompt Library: (Link to team's prompt library — Google Drive)

4.3 Human-in-the-Loop Interventions

Despite high overall quality from AI-generated code, the team documented the following categories of manual interventions:

1. RLS Policy Gaps

Several early migrations generated by the AI included `SELECT` policies but omitted `INSERT` or `UPDATE` policies needed for specific flows. For example, the initial `session_members` RLS policy allowed members to read their own rows but did not allow the session creator to insert rows for invited users. Detected during integration testing; fixed by manually amending the migration and re-running.

2. Google Places Field Mask Misconfiguration

The initial `lib/yelp.ts` implementation used a field mask that included `places.photos`, which is a nested resource requiring a separate API call pattern. The agent assumed it could be fetched inline. This caused all restaurant searches to return no photo URLs. The team manually removed `places.photos` from the mask and updated the `image_url` field to return `null`, resolving the issue.

3. Match Trigger Race on Session Completion

The `check_for_match()` trigger and the `maybe_complete_session()` function were initially generated as two separate triggers, both firing on swipe insert, with no guaranteed execution order. Under concurrent load, `maybe_complete_session` could run before `check_for_match`, closing the session before the final match was recorded. The team redesigned this so `maybe_complete_session` is called from the API route handler (after the swipe insert commit) rather than as a trigger, serializing the two operations.

4. Expo Router Deep-Link Configuration

The AI-generated lobby screen used `router.push('session/[id]/lobby')` with a string literal rather than a typed `href` object. This caused navigation to fail on Android due to path encoding differences. The team manually updated all navigation calls to use the typed `{pathname, params}` object form required by Expo Router v4.

5. JWT Refresh Timing on Mobile

The `AuthContext` implementation generated by the AI set up a Supabase `onAuthStateChange` listener but did not handle the `TOKEN_REFRESHED` event to update the stored session. On long-running sessions (>1 hour), API calls began returning 401. The team added the missing event handler to re-store the refreshed JWT.

6. Centroid Calculation for Session Location

The AI initially placed the centroid calculation inside the mobile client, sending only the computed lat/lng to the API. The team redesigned this to store each user's individual location in the session's `user_locations` JSONB column (via the `store_user_location` RPC) and compute the centroid server-side at session-start time, so late-joining members' locations could be incorporated.

4.4 Codebase

The BiteBuddy source code is hosted on GitHub under the `BiteBuddy-CS4485` organization.

- **Repository:** https://github.com/BiteBuddy-CS4485/BiteBuddy
- **Main README:** `README.md` at the repository root
- **API README:** `apps/api/README.md`
- **Mobile README:** `apps/mobile/README.md`
- **Shared Package:** `packages/shared/`
- **Database Migrations:** `supabase/migrations/`
- **Architectural Instructions:** `CLAUDE.md` at the repository root

5. System Performance & Validation

5.1 Testing and Validation

Unit Testing (Vitest)

The team wrote unit tests covering the two most algorithmic components of the system:

`apps/api/src/lib/__tests__/yelp.test.ts` — 22 tests covering the Google Places client:

- `mapPriceFilter()`: 12 tests verifying that price symbols ($, $$, $$$, $$$$) map correctly to Google Places price level enums, that empty and null inputs return `undefined`, and that invalid symbols are handled gracefully.
- `searchRestaurants()`: 10 tests covering successful restaurant fetch and field mapping, category filtering (only restaurant/cafe/bakery/fast_food/food_court primaries are included), radius capping at 50,000 meters, graceful handling of missing optional fields (phone, rating, price level), and HTTP error propagation.

`packages/shared/src/__tests__/api.test.ts` — 5 tests verifying the `CUISINE_CATEGORIES` constant: correct structure, uniqueness of aliases, presence of required fields, and that the "all" category is included.

`packages/shared/src/__tests__/geolocation.test.ts` — Tests for `calculateCentroid` (averaging coordinate arrays) and `calculateDistance` (Haversine formula accuracy against known city-pair distances).

Integration & Manual Testing (Postman)

A Postman collection was maintained covering the full session lifecycle:

1. Sign up two users (User A, User B)
2. User A searches for User B and sends a friend request
3. User B accepts the request
4. User A creates a session, invites User B
5. User B joins via invite code
6. Session is started (restaurants fetched)
7. Both users submit swipes in interleaved order
8. Match is verified in the response body and in the Supabase dashboard
9. Session completion is verified

Each PR that touched API routes required the Postman collection to be run successfully before merging.

Device Testing

End-to-end tests were performed on physical devices:

- iOS 18 (iPhone 15 Pro) via Expo Go and development build
- Android 14 (Pixel 7) via Expo Go

TypeScript Compilation

Because the entire codebase is TypeScript with strict mode enabled, `tsc -noEmit` served as a first-pass integration test across package boundaries. Type errors in the shared package surfaced immediately in both the API and mobile builds.

5.2 Technical Performance Metrics

All measurements were taken against the production Supabase instance from a device connected to university Wi-Fi.

Table 5: Performance Metrics

Metric	Value	Notes
Login response time	∼180 ms (p50)	Supabase Auth GoTrue; JWT signing overhead
Session create (no restaurants)	∼120 ms	Single DB insert
Session start (restaurant fetch)	∼1.4 s	Google Places API round-trip + 20 row inserts
Swipe response time	∼95 ms (p50)	DB insert + trigger + RPC
Match swipe response time	∼130 ms	Same path + match insert in trigger
Friends search	∼80 ms	Username `ILIKE` query with index
Session results	∼110 ms	JOIN across swipes + matches
Supabase Realtime event delivery	∼60–120 ms	WebSocket latency from Dallas region
App cold start (Expo dev build)	∼3.2 s	JavaScript bundle load
API cold start (Vercel serverless)	∼400 ms (first)	Subsequent: ∼50 ms

The dominant latency on the critical user path (swiping) is network round-trip time. The swipe trigger and RPC add less than 10 ms of database processing time, as confirmed by `EXPLAIN ANALYZE` on the trigger queries. The `session_restaurants`, `swipes`, and `matches` tables are indexed on `(session_id)` and `(session_id, restaurant_id)` (migration 013), ensuring $O(1)$ lookup for the match trigger's `COUNT` queries.

5.3 AI Model Performance

BiteBuddy does not include a user-facing AI/ML model as a product feature. The AI involvement was entirely in the build process (Claude Code as a coding agent, described in Section 4.2). Accordingly, this section is not applicable.

6. Lessons Learned

6.1 Insights Gained

The most significant insight from this project was that the quality of AI-generated code is almost entirely a function of the quality of the specification given to the agent. When the prompt was vague ("add a way for users to leave sessions"), the output required substantial revision — missing edge cases, incorrect RLS policies, incomplete error handling. When the prompt was precise — citing exact table names, column types, constraint names, and response shapes from the shared types package — the output was production-quality on the first generation and required only light review.

This represents a fundamental shift from traditional coding practice. The engineering skill that mattered most was not typing speed or library memorization but the ability to decompose a product requirement into a complete, unambiguous technical specification. "*Spec quality is code quality*" became an informal team principle.

A secondary insight was that database-layer correctness is more important than ever in AI-assisted development. The AI tended to generate correct application logic but was more likely to miss subtle concurrency or RLS edge cases in database artifacts. Investing time in careful migration review — treating each SQL file with the same scrutiny as a security-critical code review — prevented several classes of bugs that would have been very difficult to diagnose in production.

6.2 Lessons from Challenges

Challenge 1: AI Context Window Limitations for Large Features

When implementing the complete swipe + match + session-completion flow as a single prompt, the AI's output became inconsistent — it would correctly implement the swipe insert but then generate a session-completion check that referenced columns that didn't exist in the actual schema. **Solution:** Break large features into the smallest possible independently verifiable units. The swipe insert, the trigger logic, and the `maybe_complete_session` function were each implemented and tested in separate sessions.

Challenge 2: RLS Policies Are Not Obvious to AI Agents

The AI consistently generated correct SQL for table structure but required explicit prompting to produce correct RLS policies. It tended to default to `auth.uid() = user_id` patterns, which are correct for user-owned tables but wrong for shared resources like sessions (where any member should be able to read) or matches (readable by all session members). **Solution:** Include the desired RLS policy in prose in every migration prompt: "Members of a session should be able to read all swipes in that session, not just their own."

Challenge 3: Expo Router v4 Breaking Changes

The AI's training data included Expo Router v3 patterns. Several generated navigation calls and layout configurations were v3-style and failed silently or crashed on v4. **Solution:** Explicitly state the Expo Router version in every mobile-side prompt and include a v4 code snippet from the official documentation as a reference.

Challenge 4: Google Places (New) API vs. Legacy Places API

The Google Places API underwent a major version change (legacy vs. New/v1) with a different base URL, field mask pattern, and response schema. AI training data predominantly covered the legacy API. **Solution:** Provide the complete `places:searchNearby` request and response schema as part of the prompt, rather than asking the AI to "use the Google Places API."

Challenge 5: Supabase Realtime Subscription Cleanup

The AI-generated Realtime subscriptions in React Native components were not always cleaned up on component unmount, causing memory leaks and stale event listeners when navigating away and back to swipe screens. **Solution:** Establish a team convention of always returning a cleanup function from `useEffect` that calls `supabase.removeChannel(channel)`, and add this pattern to `CLAUDE.md` as a required practice.

6.3 Skills Developed

AI Engineering & Prompt Craft Every team member developed the ability to write structured, constraint-rich prompts that reliably produce correct code. This included learning how to reference existing code artifacts (file paths, function signatures, type names) rather than describing desired behavior abstractly.

Full-Stack TypeScript Development Working across a Next.js API, an Expo mobile client, and a shared TypeScript package simultaneously gave every member exposure to the complete web/mobile stack.

Database Design & PostgreSQL Writing 13 migration files that incrementally built a production-quality schema — with RLS policies, triggers, custom functions, and performance indexes — provided hands-on experience with PostgreSQL features that go well beyond basic CRUD.

Distributed Systems Thinking Managing the race condition between concurrent swipes, understanding Supabase Realtime's eventual-delivery guarantees, and handling JWT expiry in a long-lived mobile session all required reasoning about distributed system behavior.

Code Review as a Safety Net With AI generating large amounts of code rapidly, effective code review became more important, not less. The team developed discipline around reviewing AI output as carefully as hand-written code, specifically looking for the AI's known blind spots: RLS policy completeness, foreign key constraints, error response codes, and cleanup of async resources.

6.4 Individual Reflection

Neon Zheng

Producing the system's high-level architecture diagrams — designed specifically to be legible to AI coding agents — pushed me to think about system design at a level of precision I had not previously attempted, where every component boundary and data flow had to be explicit

and unambiguous. I also conducted a cybersecurity review of the application, identifying common risks such as JWT mishandling, RLS policy gaps, and injection vectors, which gave me a more security-conscious lens that I now apply by default when reviewing any backend code. This combination of architecture and security work taught me that a well-designed system and a secure system are largely the same thing.

Rafid Nasery

Leading weekly team meetings and maintaining the project's SPEC.md as its single source of truth gave me direct experience in technical project management — specifically how much a well-structured specification reduces ambiguity and rework downstream. Managing the GitHub Projects board for task delegation taught me to break work into atomic, assignable units that could be handed off to both teammates and AI agents without losing context. I also built out the CI/CD pipeline and API contract generation process, which showed me how investment in automation early in a project pays compounding dividends as the team scales up velocity. I also worked on PostgreSQL Triggers for synchronization issues that came up, which was great for learning about the use of WebSockets.

Jonathan Lee

Integrating the Google Places API taught me how to navigate real-world third-party API constraints — field masks, billing by field, and response schema differences between API versions — which required careful reading of documentation rather than relying on prior assumptions. Working through the PostgreSQL trigger and RPC flow for match detection gave me a much deeper appreciation for database-layer logic and how to design atomic operations that eliminate race conditions. Collaborating with Claude Code on scalable architecture also showed me how to write code that is structured for an AI to extend correctly, which is a skill I expect to carry into professional work.

Billy Nguyen

This project deepened my understanding of the full mobile deployment lifecycle — from configuring Expo EAS builds to running automated UI tests with Selenium and validating behavior across real devices. I also took ownership of setting up Supabase's observability tooling, which gave me hands-on experience monitoring database performance and diagnosing slow queries in a live system. Seeing how instrumentation catches issues that unit tests miss has fundamentally changed how I think about production readiness.

Abdala Aljewarane

Working with this team taught me a lot. I took ownership of building the MVP, making pragmatic scope decisions to ship quickly without boxing us in later. Setting up the API pushed me to think carefully about clean contracts — endpoints, error handling, and auth — which paid off as the project grew. Deployment was where it all came together. Moving from "works on my machine" to a live environment sharpened my thinking around reliability and real-world use. Most of all, I valued how collaborative the process was: shipping end-to-end alongside teammates whose feedback consistently made the work better than I could have made it alone.

Paola Reyes-Almendares

Using Figma and Figma AI to produce the app's wireframes gave me practical experience translating product requirements into concrete visual specifications before a single line of code was written. I also developed my ability to communicate design intent clearly to developers — learning that an annotated wireframe with spacing, interaction states, and component labels dramatically reduces back-and-forth during implementation. Most importantly, this project reinforced for me that documentation is not overhead but a core deliverable: every design decision I wrote down became a reference that kept the team aligned across weeks of parallel development.

7. Conclusion & Future Work

7.1 Summary of Key Accomplishments

BiteBuddy successfully delivered a fully functional group restaurant decision application within a single semester. The core product loop — create a session, invite friends, swipe on restaurants, receive a real-time match — works end-to-end on iOS and Android. The technical implementation achieved its primary engineering goals: atomic match detection without race conditions, real-time lobby and swipe-progress updates, type-safe end-to-end TypeScript, and a clean API architecture that enforces authorization at the database layer through Row Level Security.

Beyond the core loop, the team delivered a complete user-facing product including authentication with password reset, friend management with search and request flows, avatar photo upload, session history, recent matches, invite codes for out-of-band session sharing, centroid-based location aggregation, and session cancellation and leave flows.

The project demonstrated that AI-assisted development — when paired with a precise specification, disciplined prompt engineering, and rigorous human review — can produce production-quality code at significantly higher velocity than traditional development. The team's investment in `CLAUDE.md` and the project spec as shared artifacts paid compounding returns throughout the semester: time spent on specification reduced time spent on correction.

7.2 Future Enhancements

Near-Term (0–3 months)

- **Push Notifications (APNs/FCM):** Integrating Expo Push Notifications would allow users to be notified of matches even when the app is backgrounded.
- **Restaurant Photos:** Adding photo thumbnails to swipe cards via the `places.photos` resource would significantly improve the swipe UX.
- **In-App Messaging:** A lightweight chat channel per session (built on Supabase Realtime) would let group members coordinate within the app.
- **Tie-breaking UI:** When a session ends with multiple matches, a secondary "group vote

on the matches" round would give the final selection additional legitimacy.

Medium-Term (3–6 months)

- **Preference Learning:** Track each user's swipe history across sessions to build a preference profile (cuisine type, price level, rating threshold) for personalized restaurant ordering.
- **Restaurant Details Deep-Dive:** Integrate Google Places `getPlace` detail fetches to show hours, reviews, website, and a map view on a restaurant detail sheet.
- **Web Client:** A Next.js web frontend (sharing the existing API and shared types package) would allow users to manage sessions from a desktop browser.
- **Session Templates:** Allow users to save session configurations (location, radius, price, cuisine) as named templates for frequently recurring group dining contexts.

Long-Term (6+ months)

- **Multi-Round Swiping:** If no match is found in the first round, automatically fetch a second set of restaurants and continue swiping rather than requiring session recreation.
- **Social Graph Expansion:** Add group profiles, recurring "dining groups," and the ability to invite non-app users via SMS/link who join via a web-based guest swipe flow.
- **Reservation Integration:** Partner with OpenTable or Resy APIs to allow direct reservation booking from the match results screen.
- **Analytics Dashboard:** A personal "dining history" screen showing most-matched cuisines, most-swiped-right restaurants, and friend match compatibility scores.

References

1. Supabase Documentation — Authentication, Database, Realtime, Storage. https://supabase.com/docs
2. Google Places API (New) Documentation — Nearby Search. https://developers.google.com/maps/documentation/places/web-service/nearby-search
3. Next.js 16 Documentation — App Router, Route Handlers. https://nextjs.org/docs
4. Expo Documentation — Expo Router v4, Expo Location, Expo Image Picker. https://docs.expo.dev
5. React Native Documentation — Core Components and APIs. https://reactnative.dev/docs
6. Vitest Documentation — Unit Testing Framework. https://vitest.dev
7. Supabase Row Level Security — Policies and Auth Helpers. https://supabase.com/docs/guides/auth/row-level-security

8. PostgreSQL Documentation — Trigger Functions, PL/pgSQL. https://www.postgresql.org/docs/15/plpgsql-trigger.html
9. Anthropic Claude Code — AI Coding Assistant. https://claude.ai/code
10. React Native Reanimated Documentation. https://docs.swmansion.com/react-native-reanimate
11. React Native Gesture Handler Documentation. https://docs.swmansion.com/react-native-gestu

Acknowledgements

The BiteBuddy team thanks Professor Sridhar Alagar for his guidance and feedback throughout the semester and Project Coordinator Adarsh Gella for keeping the project organized and on schedule. We thank the open-source maintainers of Supabase, Expo, and the React Native ecosystem whose work made this project possible in its scope and timeline. We also acknowledge Anthropic's Claude Code, which served as our primary AI engineering assistant and substantially accelerated our development velocity.